Maths and ML

Every formula with a worked example, simple charts and honest limits

CoolRoute: Pedestrian Routing with Shade and Heat-Exposure Budgets

Every formula here is the one used in the code, with a worked example. Symbols are explained in plain words.

1. Sun strength

How strong is the sun at this hour? We use the angle of the sun above the horizon:

```
sun(t) = max(0, sin(phi) x sin(delta) + cos(phi) x cos(delta) x cos(h))
```

- **phi** = latitude (Jaipur 26.9 degrees)
- **delta** = sun's tilt for the date (about -10 degrees in mid-October)
- **h** = hour angle = 15 degrees x (hour - 12), so it is 0 at noon
- Result is 0 at night and about 0.8 at midday, shown as a 0 to 1 strength

Hour	7:00	9:00	11:00	13:00	15:00	17:00	19:00
sun(t)	0.15	0.55	0.77	0.77	0.55	0.15	0.00

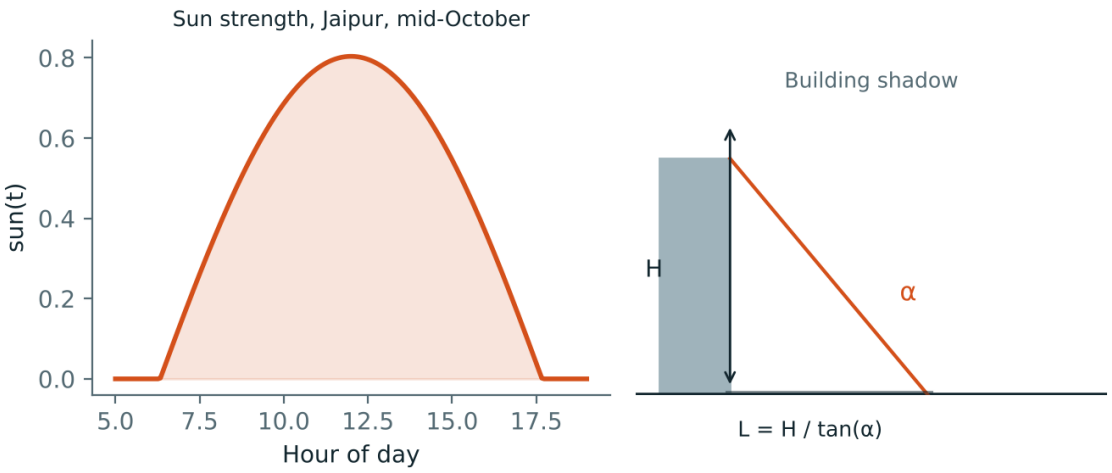


Figure 1. Left: sun strength through the day. Right: a building of height H casts a shadow of length L = H / tan(alpha).

2. Heat exposure of one street piece

```
heat = length x (1 - shade) x sun(t)  +  wait x walking_speed x (1 - crossing_shade) x sun(t)
```

Think of it as **metres walked in full sun**. A 100 m street with 80% shade at full sun is the same as 20 m in the sun. The second part counts the time you stand at a crossing: you stay where you are, so we count the distance you would have walked in that time (walking speed 1.3 m/s).

Worked example (at 1 pm, sun = 0.774)

```
Street piece: length 82 m, shade 0.20, crossing wait 35 s, crossing shade 0.10
walking heat = 82 x (1 - 0.20) x 0.774          = 50.8
waiting heat = 35 x 1.3 x (1 - 0.10) x 0.774    = 31.7
heat of the piece                                = 82.5
```

3. Effort and time

```
effort = length x (1 + 0.05 x slope%)        # a 6% slope makes 100 m feel like 130 m
time   = length / 1.3 + wait                 # seconds
```

4. The optimisation problem

We want the shortest route that stays inside the heat budget:

```
minimise   distance(route)
subject to heat(route) <= budget        budget = (budget% / 100) x heat(shortest route)
           every segment on the route is step-free   (if requested)
           no closed footpath on the route
```

This is a **constrained shortest path**. Solving it exactly is hard (it is NP-hard in general), so we use a fast, well-known method called **Lagrangian relaxation**.

Lagrangian relaxation in plain words

Give heat a price, lambda. Each street piece then costs **distance + lambda x heat**. With lambda = 0, the cheapest route is the shortest one. As lambda grows, hot streets become expensive and the route moves to shade. We search for the smallest lambda whose route fits the budget.

```
cost(edge) = distance(edge) + lambda x heat(edge)
```

Why bisection works: as lambda goes up, the heat of the best route never goes up, and its distance never goes down. So if one lambda is too hot we move up, and if it is cool enough we try lower. 40 steps pin lambda down very closely.

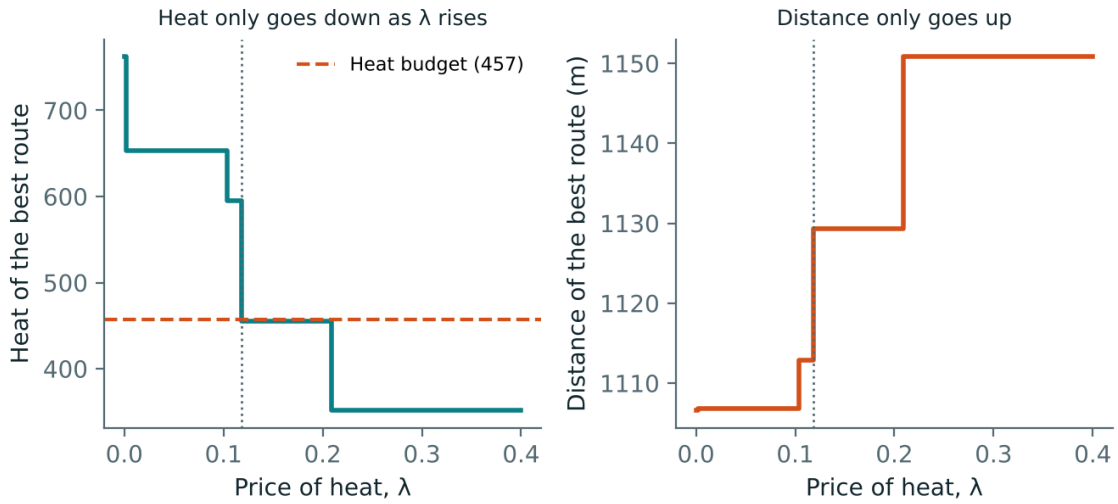


Figure 2. As lambda rises, heat falls and distance grows, in steps. The search picks the smallest lambda whose heat is under the budget line (dotted vertical line).

Honest note: this method finds a route on the efficient frontier, and it always fits the budget when one exists on that frontier. It may not be the single shortest route among all that fit, but it is close and very fast. An exact method (label-setting) is a planned upgrade.

The trade-off curve

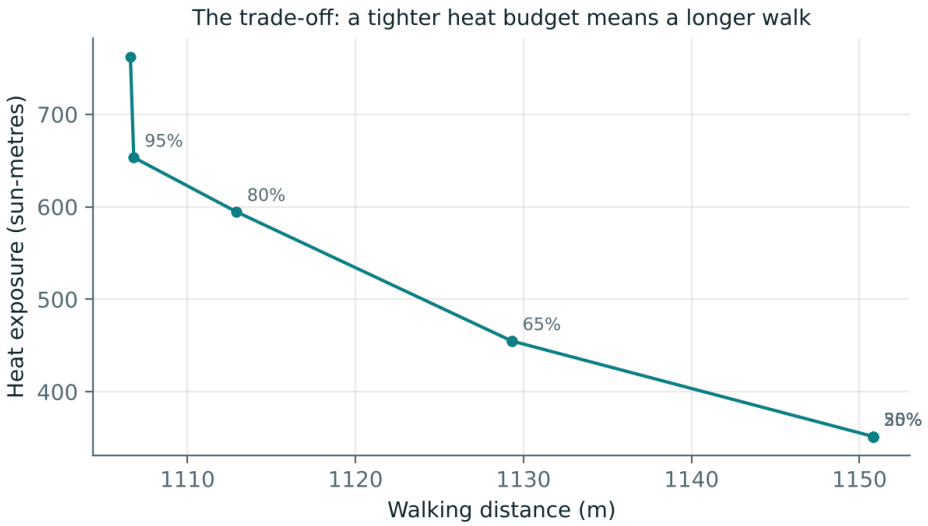


Figure 3. Tighter budgets cost more distance. This is the picture to show a judge.

Budget	Distance (m)	Heat	Budget met?
30%	1151	352	no, coolest route shown
40%	1151	352	no, coolest route shown
50%	1151	352	yes
60%	1129	455	yes
70%	1129	455	yes
80%	1113	595	yes
100%	1107	762	yes

5. Shade from geometry

For a street with building height H, width W, at sun elevation alpha and sun direction gamma, with the street running along bearing theta:

```
shadow length          L        = H / tan(alpha)
shadow across street   L_perp   = L x |sin(gamma - theta)|
building shade         s_b      = min(1, L_perp / W)
tree shade (from NDVI) s_t      = clip(1.4 x NDVI - 0.1, 0, 0.9)
combined shade         s        = 1 - (1 - s_b) x (1 - s_t)
```

The last line says: a spot is sunny only if it is missed by **both** the building shadow and the tree shadow.

6. Machine learning for shade and confidence

Real cities have gaps: unknown building heights, old tree data. A model learns shade from features and also says how sure it is.

Item	Choice	Why
Inputs	building height, street width, street bearing, tree greenness (NDVI), sun elevation, sun azimuth	All can be taken from open data
Model	Gradient-boosted trees	Works well on small tables, fast, no GPU needed
Output	Shade (middle estimate) and a range from the 10th to the 90th percentile	The range becomes the confidence value

Item	Choice	Why
Loss for the range	Pinball (quantile) loss	Teaches the model to give honest lower and upper bounds

```
pinball loss (quantile q):   L_q = q x (y - y_hat)      if y >= y_hat
                                   (1 - q) x (y_hat - y) if y <  y_hat
MAE = average of |y - y_hat|        R2 = 1 - sum (y - y_hat)^2 / sum (y - mean(y))^2
confidence = 1 - (p90 - p10)        # narrow range -> high confidence
rule: confidence < 0.3  ->  draw the street dashed and warn the user
```

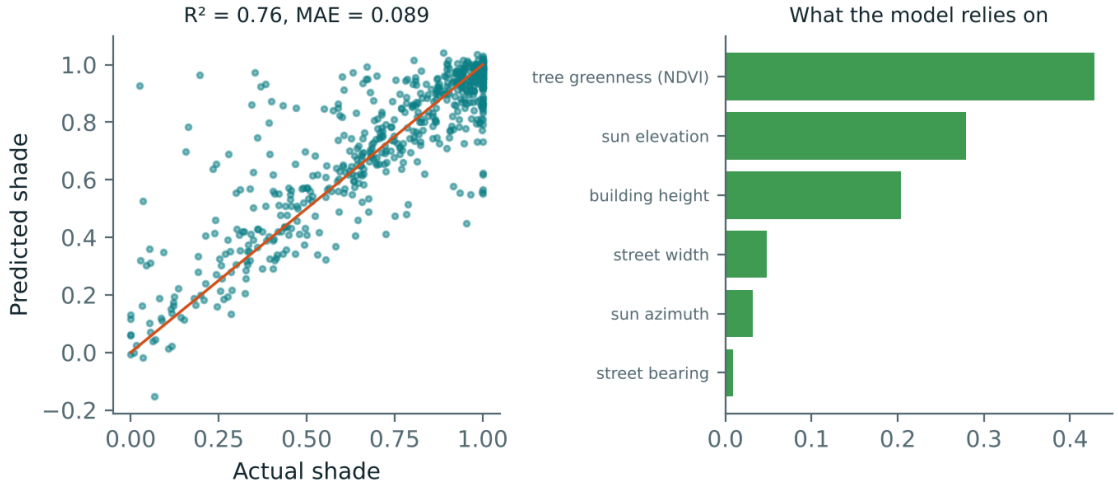


Figure 4. Left: predicted versus actual shade on held-out data. Right: which inputs the model uses most.

Measure (held-out data)	Value
Training samples	6,000 (25% held out)
R2	0.76
Mean absolute error	0.089 (on a 0 to 1 shade scale)
Share of true values inside the 10-90 range	79% (target 80%)
Top inputs	tree greenness (NDVI) (43%), sun elevation (28%), building height (20%)

Be honest with judges: this model was trained on synthetic data made from the shadow formula above, plus noise. It shows that the pipeline and the confidence idea work. It does not show accuracy on real Jaipur streets. Real accuracy needs real tree and shade data, which is the first item on the roadmap.

7. Complexity summary

Step	Cost
One shortest-path search	O(E log V)
Heat-budget search	up to 41 searches
Our prototype graph	64 junctions, 112 street pieces: milliseconds
Real city	Add a bounding box around A and B, or contraction hierarchies